Figure 1.1 An API handles requests from clients on behalf of users. Clients may be web browsers, mobile apps, devices in the Internet of Things, or other APIs. The API services requests according to its internal logic and then at some point returns a response to the client. The implementation of the API may require talking to other "backend" APIs, provided by databases or processing systems.

A UI also provides a boundary to a software system and restricts the operations that can be performed. What distinguishes an API from a UI is that an API is explicitly designed to be easy to interact with by other software, while a UI is designed to be easy for a user to interact with directly. Although a UI might present information in a rich form to make the information pleasing to read and easy to interact with, an API typically will present instead a highly regular and stripped-back view of the raw data in a form that is easy for a program to parse and manipulate.

This book will focus on APIs exposed over HTTP using a loosely RESTful approach, as this is the predominant style of API at the time of writing. That is, while the APIs that are developed in this book will try to follow REST design principles, you will sometimes deviate from those principles to demonstrate how to secure other styles of API design. Much of the advice will apply to other styles too, and the general principles will even apply when designing a library.

1.3 API security in context